Lecture_2

2025-05-27

Created by Dr. Preeti Jagadev

Activity 1

Creating a Dataframe in R

```
myFamilyNames <- c("Dad", "Mom", "Sis", "Bro", "Dog")
myFamilyNames
```

```
## [1] "Dad" "Mom" "Sis" "Bro" "Dog"
```

```
myFamilyAges <- c(43, 42, 12, 8, 5)
myFamilyAges
```

```
## [1] 43 42 12  8  5
```

```
myFamilyGenders <- c("Male", "Female", "Female", "Male", "Female")
myFamilyGenders
```

```
## [1] "Male"   "Female" "Female" "Male"   "Female"
```

```
myFamilyWeights <- c(188, 136, 83, 61, 44)
myFamilyWeights
```

```
## [1] 188 136  83  61  44
```

```
myFamily <- data.frame(myFamilyNames, myFamilyAges, myFamilyGenders, myFamilyWeights)
myFamily
```

```
##   myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
## 1           Dad           43            Male             188
## 2           Mom           42          Female             136
## 3           Sis           12          Female              83
## 4           Bro            8            Male              61
## 5           Dog            5          Female              44
```

```
str(myFamily)
```

```
## 'data.frame':    5 obs. of  4 variables:
##  $ myFamilyNames  : chr  "Dad" "Mom" "Sis" "Bro" ...
##  $ myFamilyAges   : num  43 42 12 8 5
##  $ myFamilyGenders: chr  "Male" "Female" "Female" "Male" ...
##  $ myFamilyWeights: num  188 136 83 61 44
```

What does the structure function tell us about the data object myFamily?

Confirmation that MyFamily is a data frame;

MyFamily has five observations (cases/instances) and four variables.

"$" for each variable column with descriptive information.

"num" variable indicates "numeric".

"chr" variable indicates "character"

```r
summary(myFamily)
```

```
##  myFamilyNames       myFamilyAges myFamilyGenders    myFamilyWeights
##  Length:5           Min.   : 5    Length:5           Min.   : 44.0
##  Class :character   1st Qu.: 8    Class :character   1st Qu.: 61.0
##  Mode  :character   Median :12    Mode  :character   Median : 83.0
##                     Mean   :22                       Mean   :102.4
##                     3rd Qu.:42                       3rd Qu.:136.0
##                     Max.   :43                       Max.   :188.0
```

Let's Understand Each Column's Summary

myFamilyNames

Length: 5 (There are 5 names in this list (i.e., 5 people in myFamily)).

Class: character (This column stores text values).

Mode: character (This confirms the type of data is character/text).

So, R is saying: "This is a list of 5 names, and all values are text."

myFamilyAges

Min.: 5 (The youngest family member is 5 years old).

1st Qu.: 8 (25% of the ages are below or equal to 8. (This is the first quartile)).

Median: 12 (The middle age (when sorted) is 12).

Mean: 22 (The average age of myfamily is 22 years).

3rd Qu.: 42 (75% of the ages are below or equal to 42).

Max.: 43 (The oldest person is 43 years old).

This gives you an idea of the spread and center of the ages in myfamily.

myFamilyGenders

Length: 5 (You have 5 gender entries (one for each person)).

Class: character (The gender values are stored as text (like "Male", "Female")).

Mode: character (Again confirms that it's text data).

So this column holds the gender of each person in the family.

myFamilyWeights

Min.: 44.0 (The lightest person weighs 44 kg (or another unit)).

1st Qu.: 61.0 (25% of the weights are below or equal to 61.0).

Median: 83.0 (The middle weight is 83.0).

Mean: 102.4 (The average weight is 102.4).

3rd Qu.: 136.0 (75% of the weights are below or equal to 136.0).

Max.: 188.0 (The heaviest person weighs 188.0).

Again, this shows the range and average of weights in myfamily.

Accessing Dataframes

Accessing Dataframes as a vector:

```
myFamily$myFamilyAges[2]
```

```
## [1] 42
```

```
#Returns the second element in myFamilyAges
```

Accessing Dataframes as a matrix:

```
myFamily[1,1]
```

```
## [1] "Dad"
```

```
#Returns the data element in the first row and first column
```

```
myFamily[1,]
```

```
##   myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
## 1           Dad           43            Male             188
```

```
#Returns the first row
```

```
myFamily[,1]
```

```
## [1] "Dad" "Mom" "Sis" "Bro" "Dog"
```

```
#Returns the first column
```

```
myFamily[-1,]
```

```
##   myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
## 2           Mom           42          Female             136
## 3           Sis           12          Female              83
## 4           Bro            8            Male              61
## 5           Dog            5          Female              44
```

```
#Returns everything but the 1st row  (i.e., deletes the 1st  row)
```

```
myFamily[,-1]
```

```
##   myFamilyAges myFamilyGenders myFamilyWeights
## 1           43            Male             188
## 2           42          Female             136
## 3           12          Female              83
## 4            8            Male              61
## 5            5          Female              44
```

```
#Returns everything but the 1st column (i.e., deletes the 1st column)
```

Activity 2

"mtcars" is a built-in dataset in R. It's a table (data frame) where: Rows are different car models, and Columns are different measurements (like miles per gallon, horsepower, etc.)

How to Sort a Data Frame Correctly

If you want to sort the "mtcars" data frame, you need to:

Pick a specific column to sort by.

For example, suppose you want to sort by the miles per gallon (mpg) column. You do it like this:

```
mtcars[order(mtcars$mpg), ]
```

```
##                      mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
## Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
## Camaro Z28          13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
## Duster 360          14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
## Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
## Maserati Bora       15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
## Merc 450SLC         15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
## AMC Javelin         15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
## Dodge Challenger    15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
## Ford Pantera L      15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
## Merc 450SE          16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
## Merc 450SL          17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
## Merc 280C           17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Valiant             18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## Hornet Sportabout   18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
## Merc 280            19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Pontiac Firebird    19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
## Ferrari Dino        19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
## Mazda RX4           21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag       21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Hornet 4 Drive      21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Volvo 142E          21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
## Toyota Corona       21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Datsun 710          22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Merc 230            22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Merc 240D           24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Porsche 914-2       26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Fiat X1-9           27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Honda Civic         30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Lotus Europa        30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Fiat 128            32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Toyota Corolla      33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
```

This sorts the rows of "mtcars" in ascending order based on the "mpg" column.

"order(mtcars$mpg)" gives the row numbers in the correct sorted order.

"mtcars[...]" rearranges the rows based on those row numbers.

If you want descending order, use:

```
mtcars[order(-mtcars$mpg), ]
```

```
##                      mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Toyota Corolla      33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Fiat 128            32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
```

```
## Honda Civic         30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Lotus Europa        30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Fiat X1-9           27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2       26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Merc 240D           24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Datsun 710          22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Merc 230            22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Toyota Corona       21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Hornet 4 Drive      21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Volvo 142E          21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
## Mazda RX4           21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag       21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Ferrari Dino        19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
## Merc 280            19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Pontiac Firebird    19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
## Hornet Sportabout   18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
## Valiant             18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## Merc 280C           17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Merc 450SL          17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
## Merc 450SE          16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
## Ford Pantera L      15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
## Dodge Challenger    15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
## Merc 450SLC         15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
## AMC Javelin         15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
## Maserati Bora       15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
## Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
## Duster 360          14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
## Camaro Z28          13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
## Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
## Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
```

```r
indexes <- order(mtcars[,"mpg"])
```

In the above chunk: "mtcars[,"mpg"]" extracts the "mpg" column (miles per gallon).

"order(...)": Returns the row numbers that would sort the data in increasing order of "mpg".

You save those row numbers in indexes.

Look at the first index

```r
indexes[1]
```

```
## [1] 15
```

This tells you: the 15th row of "mtcars" has the lowest "mpg" (i.e., it is the least fuel-efficient car).

Look at that row

```r
mtcars[15,]
```

```
##                     mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Cadillac Fleetwood 10.4   8  472 205 2.93 5.25 17.98  0  0    3    4
```

This shows the entire 15th row of the "mtcars" data frame.This car has an "mpg" of 10.4, which is very low — meaning it consumes a lot of fuel.

Sort the full data frame

```
sortedDF <- mtcars[indexes,]
```

This line rearranges the entire data frame "mtcars" in order of increasing "mpg", using the sorted row numbers from indexes.

The result is stored in "sortedDF".

Now "sortedDF" starts with the car that has the lowest "mpg" and ends with the one that has the highest "mpg".

Exploring the DataFrame

```
str(mtcars)
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
##  $ disp: num  160 160 108 258 360 ...
##  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
##  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ qsec: num  16.5 17 18.6 19.4 17 ...
##  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
##  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
##  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
##  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

Exploring the dataframe the tidyverse way

```
install.packages("tidyverse")
```

```
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.4'
## (as 'lib' is unspecified)
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
glimpse(mtcars)
```

```
## Rows: 32
## Columns: 11
## $ mpg  <dbl> 21.0, 21.0, 22.8, 21.4, 18.7, 18.1, 14.3, 24.4, 22.8, 19.2, 17.8,~
## $ cyl  <dbl> 6, 6, 4, 6, 8, 6, 8, 4, 4, 6, 6, 8, 8, 8, 8, 8, 8, 4, 4, 4, 4, 8,~
## $ disp <dbl> 160.0, 160.0, 108.0, 258.0, 360.0, 225.0, 360.0, 146.7, 140.8, 16~
```

```
## $ hp   <dbl> 110, 110, 93, 110, 175, 105, 245, 62, 95, 123, 123, 180, 180, 180~
## $ drat <dbl> 3.90, 3.90, 3.85, 3.08, 3.15, 2.76, 3.21, 3.69, 3.92, 3.92, 3.92,~
## $ wt   <dbl> 2.620, 2.875, 2.320, 3.215, 3.440, 3.460, 3.570, 3.190, 3.150, 3.~
## $ qsec <dbl> 16.46, 17.02, 18.61, 19.44, 17.02, 20.22, 15.84, 20.00, 22.90, 18~
## $ vs   <dbl> 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0,~
## $ am   <dbl> 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0,~
## $ gear <dbl> 4, 4, 4, 3, 3, 3, 3, 4, 4, 4, 4, 3, 3, 3, 3, 3, 3, 4, 4, 4, 3, 3,~
## $ carb <dbl> 4, 4, 1, 1, 2, 1, 4, 2, 2, 4, 4, 3, 3, 3, 4, 4, 4, 1, 2, 1, 1, 2,~
```

Selecting a Column

```r
mtcars[ , "mpg"]
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

This is saying: "Give me the column named 'mpg' from the mtcars dataset."

It uses bracket notation: data[rows, columns]

Leaving the row part empty ([,]) means "all rows"

"mpg" tells R which column to get

So it returns all the miles-per-gallon (mpg) values.

```r
mtcars[ , 1]
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

This is also using bracket notation, but it asks for column number 1

It returns the first column, which in mtcars is"mpg"

```r
mtcars$mpg
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

This is dollar-sign notation: data$column_name

It's a shortcut to access a column directly by its name

It returns the 'mpg' column (miles per gallon)

Selecting a Column the tidyverse way

"select()" lets you choose specific columns from a dataset (like a spreadsheet).

It returns a smaller version of the dataset with only the columns you asked for.

```r
select(mtcars, mpg)
```

```
##                     mpg
## Mazda RX4          21.0
## Mazda RX4 Wag      21.0
## Datsun 710         22.8
## Hornet 4 Drive     21.4
```

```
## Hornet Sportabout   18.7
## Valiant             18.1
## Duster 360          14.3
## Merc 240D           24.4
## Merc 230            22.8
## Merc 280            19.2
## Merc 280C           17.8
## Merc 450SE          16.4
## Merc 450SL          17.3
## Merc 450SLC         15.2
## Cadillac Fleetwood  10.4
## Lincoln Continental 10.4
## Chrysler Imperial   14.7
## Fiat 128            32.4
## Honda Civic         30.4
## Toyota Corolla      33.9
## Toyota Corona       21.5
## Dodge Challenger    15.5
## AMC Javelin         15.2
## Camaro Z28          13.3
## Pontiac Firebird    19.2
## Fiat X1-9           27.3
## Porsche 914-2       26.0
## Lotus Europa        30.4
## Ford Pantera L      15.8
## Ferrari Dino        19.7
## Maserati Bora       15.0
## Volvo 142E          21.4
```

It means: From the "mtcars" dataset, give me just the "mpg" column.

"mpg" is written without quotes — the tidyverse knows you mean the column name.

What you get back: A smaller tibble/data frame with only the "mpg" column.

Selecting more than one column

```r
mtcars[ , c("mpg", "hp")]
```

```
##                      mpg  hp
## Mazda RX4           21.0 110
## Mazda RX4 Wag       21.0 110
## Datsun 710          22.8  93
## Hornet 4 Drive      21.4 110
## Hornet Sportabout   18.7 175
## Valiant             18.1 105
## Duster 360          14.3 245
## Merc 240D           24.4  62
## Merc 230            22.8  95
## Merc 280            19.2 123
## Merc 280C           17.8 123
## Merc 450SE          16.4 180
## Merc 450SL          17.3 180
## Merc 450SLC         15.2 180
## Cadillac Fleetwood  10.4 205
```

```
## Lincoln Continental 10.4 215
## Chrysler Imperial   14.7 230
## Fiat 128            32.4  66
## Honda Civic         30.4  52
## Toyota Corolla      33.9  65
## Toyota Corona       21.5  97
## Dodge Challenger    15.5 150
## AMC Javelin         15.2 150
## Camaro Z28          13.3 245
## Pontiac Firebird    19.2 175
## Fiat X1-9           27.3  66
## Porsche 914-2       26.0  91
## Lotus Europa        30.4 113
## Ford Pantera L      15.8 264
## Ferrari Dino        19.7 175
## Maserati Bora       15.0 335
## Volvo 142E          21.4 109
```

What it means: mtcars is your dataset (a data frame).

Inside the brackets:

The space before the comma is left empty: this means "all rows".

After the comma, you have c("mpg", "hp") — this means "only the columns named mpg and hp".

So you are selecting: All rows, and only the columns "mpg" (miles per gallon) and "hp" (horsepower).

```r
mtcars[ ,c( 1,4)]
```

```
##                      mpg  hp
## Mazda RX4           21.0 110
## Mazda RX4 Wag       21.0 110
## Datsun 710          22.8  93
## Hornet 4 Drive      21.4 110
## Hornet Sportabout   18.7 175
## Valiant             18.1 105
## Duster 360          14.3 245
## Merc 240D           24.4  62
## Merc 230            22.8  95
## Merc 280            19.2 123
## Merc 280C           17.8 123
## Merc 450SE          16.4 180
## Merc 450SL          17.3 180
## Merc 450SLC         15.2 180
## Cadillac Fleetwood  10.4 205
## Lincoln Continental 10.4 215
## Chrysler Imperial   14.7 230
## Fiat 128            32.4  66
## Honda Civic         30.4  52
## Toyota Corolla      33.9  65
## Toyota Corona       21.5  97
## Dodge Challenger    15.5 150
## AMC Javelin         15.2 150
## Camaro Z28          13.3 245
## Pontiac Firebird    19.2 175
## Fiat X1-9           27.3  66
```

```
## Porsche 914-2       26.0  91
## Lotus Europa        30.4 113
## Ford Pantera L      15.8 264
## Ferrari Dino        19.7 175
## Maserati Bora       15.0 335
## Volvo 142E          21.4 109
```

What It Means: mtcars is the data frame (a table of car data).

The space before the comma is empty → means all rows

After the comma: c(1, 4) → means columns 1 and 4

So, you are saying:

"Give me all the rows, but only the 1st and 4th columns."

Selecting more than one column, the tidyverse way:

```r
select(mtcars, mpg, hp)
```

```
##                      mpg  hp
## Mazda RX4           21.0 110
## Mazda RX4 Wag       21.0 110
## Datsun 710          22.8  93
## Hornet 4 Drive      21.4 110
## Hornet Sportabout   18.7 175
## Valiant             18.1 105
## Duster 360          14.3 245
## Merc 240D           24.4  62
## Merc 230            22.8  95
## Merc 280            19.2 123
## Merc 280C           17.8 123
## Merc 450SE          16.4 180
## Merc 450SL          17.3 180
## Merc 450SLC         15.2 180
## Cadillac Fleetwood  10.4 205
## Lincoln Continental 10.4 215
## Chrysler Imperial   14.7 230
## Fiat 128            32.4  66
## Honda Civic         30.4  52
## Toyota Corolla      33.9  65
## Toyota Corona       21.5  97
## Dodge Challenger    15.5 150
## AMC Javelin         15.2 150
## Camaro Z28          13.3 245
## Pontiac Firebird    19.2 175
## Fiat X1-9           27.3  66
## Porsche 914-2       26.0  91
## Lotus Europa        30.4 113
## Ford Pantera L      15.8 264
## Ferrari Dino        19.7 175
## Maserati Bora       15.0 335
## Volvo 142E          21.4 109
```

You are telling R: "From the mtcars dataset, select the columns mpg and hp."

```r
select(mtcars, 1, 4)
```

```
##                      mpg  hp
## Mazda RX4           21.0 110
## Mazda RX4 Wag       21.0 110
## Datsun 710          22.8  93
## Hornet 4 Drive      21.4 110
## Hornet Sportabout   18.7 175
## Valiant             18.1 105
## Duster 360          14.3 245
## Merc 240D           24.4  62
## Merc 230            22.8  95
## Merc 280            19.2 123
## Merc 280C           17.8 123
## Merc 450SE          16.4 180
## Merc 450SL          17.3 180
## Merc 450SLC         15.2 180
## Cadillac Fleetwood  10.4 205
## Lincoln Continental 10.4 215
## Chrysler Imperial   14.7 230
## Fiat 128            32.4  66
## Honda Civic         30.4  52
## Toyota Corolla      33.9  65
## Toyota Corona       21.5  97
## Dodge Challenger    15.5 150
## AMC Javelin         15.2 150
## Camaro Z28          13.3 245
## Pontiac Firebird    19.2 175
## Fiat X1-9           27.3  66
## Porsche 914-2       26.0  91
## Lotus Europa        30.4 113
## Ford Pantera L      15.8 264
## Ferrari Dino        19.7 175
## Maserati Bora       15.0 335
## Volvo 142E          21.4 109
```

This selects columns by their position (number) instead of name.

Column 1 = mpg

Column 4 = hp

Selecting Rows

```r
mtcars[ 1:2,]
```

```
##               mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4      21   6  160 110  3.9 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21   6  160 110  3.9 2.875 17.02  0  1    4    4
```

What It Means:

mtcars is the dataset (a table of car data).

1:2 before the comma means rows 1 and 2

The empty space after the comma means all columns

So this command says:

"Give me rows 1 and 2, and all columns from the mtcars dataset."

Selecting Rows the Tidyverse way:

```
slice(mtcars,1:2)
```

```
##               mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4      21   6  160 110  3.9 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21   6  160 110  3.9 2.875 17.02  0  1    4    4
```

What it does:

mtcars: the dataset

slice(): a function that selects specific rows (by number)

1:2: tells R to select rows 1 and 2

So this command returns the first two rows of the mtcars dataset, including all columns.

Pipe Operator:

%>% is called the "pipe" operator.

It comes from the "magrittr package", and is heavily used in tidyverse.

It helps make your code easier to read, like a sentence:

"Take this → then do this → then do that."

Why Use a Pipe?

Instead of nesting functions inside each other, you can write it using a pipe:

```
x <- c(4, 1, 10, 5)
x %>% min
```

```
## [1] 1
```

This means:

"Take x, then find the minimum."

How It Works:

The pipe takes the result of the left-hand side (x) and passes it as the first argument to the function on the right (min()).

So:

x %>% min() is the same as: min(x)

Pipes make code: Cleaner,and easier to read.

```
mtcars  %>% slice(1:2)
```

```
##               mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4      21   6  160 110  3.9 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21   6  160 110  3.9 2.875 17.02  0  1    4    4
```

mtcars: This is your dataset — a built-in table of car data.

%>%: This says "then." It's passing mtcars to the next function.

slice(1:2): This selects rows 1 and 2 from the data.

So the whole line means:

"Take the mtcars dataset, then give me the first two rows."

Filtering DataFrames

```
mtcars[mtcars$mpg > 28,  ]
```

```
##                 mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Fiat 128       32.4   4 78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4 75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4 71.1  65 4.22 1.835 19.90  1  1    4    1
## Lotus Europa   30.4   4 95.1 113 3.77 1.513 16.90  1  1    5    2
```

What it means (step by step):

mtcars: This is your dataset.

mtcars$mpg: This pulls out the mpg column (miles per gallon).

mtcars$mpg > 28: This creates a TRUE/FALSE vector that checks which cars have mpg greater than 28.

The row part is mtcars$mpg > 28, so it keeps only those rows where the condition is TRUE.

The column part is empty, so it includes all columns.

So this command means:

"Give me all rows from mtcars where mpg (miles per gallon) is greater than 28."

Filtering DataFrames the Tidyverse way

```
mtcars %>% filter(mpg > 28)
```

```
##                 mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Fiat 128       32.4   4 78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4 75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4 71.1  65 4.22 1.835 19.90  1  1    4    1
## Lotus Europa   30.4   4 95.1 113 3.77 1.513 16.90  1  1    5    2
```

What it does (in simple terms):

mtcars: The dataset (a table of car data)

%>%: The pipe operator — it passes mtcars into the next function

filter(mpg > 28): Keeps only the rows where the mpg (miles per gallon) is greater than 28

So this line means:

"From the mtcars dataset, keep only the cars with mpg greater than 28."

Adding a Column

```r
myCars <- mtcars
carName <- rownames(myCars)
myCars$carName <- carName
myCars [1,
     c("mpg", "cyl", "carName", "hp")]
```

```
##           mpg cyl   carName  hp
## Mazda RX4  21   6 Mazda RX4 110
```

Step-by-Step Explanation:

myCars <- mtcars You're creating a copy of the built-in mtcars dataset and naming it myCars.

This lets you modify it without changing the original mtcars.

carName <- rownames(myCars) In R, the row names of mtcars are the car model names (e.g., "Mazda RX4").

This line stores all those car names in a new variable called carName.

myCars$carName <- carName You are creating a new column in your data frame called carName, and assigning the car model names to it.

Now each row has the car name in a column, instead of just as a row label.

myCars[1, c("mpg", "cyl", "carName", "hp")] This selects row 1 from myCars, and returns only the columns:

"mpg": miles per gallon

"cyl": number of cylinders

"carName": the car's name (which you just added)

"hp": horsepower

So you're asking:

"From the first row, show me the mpg, cyl, car name, and hp."

Adding a Column the tidyverse way:

```r
myCars <- mtcars
myCars <- rownames_to_column(myCars, var = "carName")
myCars %>%
     slice(1) %>%
     select(mpg, cyl, carName, hp)
```

```
##   mpg cyl   carName  hp
## 1  21   6 Mazda RX4 110
```

Step-by-Step Explanation:

myCars <- mtcars You're creating a new copy of the mtcars dataset called myCars.

myCars <- rownames_to_column(myCars, var = "carName") This moves the row names (car model names like "Mazda RX4") into a new column called "carName".

Pipe sequence (%>%):

myCars %>% slice(1) %>% select(mpg, cyl, carName, hp)

This breaks down to:

slice(1) → Select only the first row select(mpg, cyl, carName, hp) → Choose only the specified columns

This means:

"Take the myCars data, get just the first car, and show only the mpg, cylinders, car name, and horsepower."

Grouping Information:

```
mean(myCars[myCars$cyl == 4, "mpg"])
```

```
## [1] 26.66364
```

Filters rows where the number of cylinders (cyl) is 4

Then selects the "mpg" column from those rows

Finally, calculates the average "mpg"

Interpretation: Cars with 4 cylinders have an average mpg of ~26.66, which is the most fuel-efficient group.

```
mean(myCars[myCars$cyl == 6, "mpg"])
```

```
## [1] 19.74286
```

Filters for 6-cylinder cars, then averages their "mpg".

Interpretation of the output: Cars with 6 cylinders have a moderate fuel efficiency — average ~19.74 mpg.

```
mean(myCars[myCars$cyl == 8, "mpg"])
```

```
## [1] 15.1
```

Filters for 8-cylinder cars, then averages "mpg"

Interpretation of the output: Cars with 8 cylinders have the lowest fuel efficiency — average ~15.1 mpg.

Grouping Information the Tidyverse style:

```
myCars %>%
  group_by(cyl) %>%
  summarize(mpg = mean(mpg), hp = mean(hp))
```

```
## # A tibble: 3 x 3
##     cyl   mpg    hp
##   <dbl> <dbl> <dbl>
## 1     4  26.7  82.6
## 2     6  19.7 122.
## 3     8  15.1 209.
```

What It Does (In Simple Words):

"myCars" This is your data frame (copied from mtcars and now includes a carName column).

%>% (pipe operator) This passes the data from one function to the next.

group_by(cyl) This tells R to group the data by the number of cylinders (4, 6, or 8). You're creating three groups based on that variable.

summarize(mpg = mean(mpg), hp = mean(hp)) For each cylinder group, you're asking R to: Calculate the average mpg (miles per gallon), and calculate the average hp (horsepower)

Sorting a DataFrame:

```r
indexes <- order(mtcars[,"mpg"])
sortedDF <- mtcars[indexes,]
str(sortedDF)
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ mpg : num  10.4 10.4 13.3 14.3 14.7 15 15.2 15.2 15.5 15.8 ...
##  $ cyl : num  8 8 8 8 8 8 8 8 8 8 ...
##  $ disp: num  472 460 350 360 440 ...
##  $ hp  : num  205 215 245 245 230 335 180 150 150 264 ...
##  $ drat: num  2.93 3 3.73 3.21 3.23 3.54 3.07 3.15 2.76 4.22 ...
##  $ wt  : num  5.25 5.42 3.84 3.57 5.34 ...
##  $ qsec: num  18 17.8 15.4 15.8 17.4 ...
##  $ vs  : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ am  : num  0 0 0 0 0 1 0 0 0 1 ...
##  $ gear: num  3 3 3 3 3 5 3 3 3 5 ...
##  $ carb: num  4 4 4 4 4 8 3 2 2 4 ...
```

Step-by-Step Explanation:

1. indexes <- order(mtcars[,"mpg"])

mtcars[,"mpg"] pulls out the mpg column (just like mtcars$mpg)

order(...) returns the row indices that would sort the mpg values in ascending order

So indexes is a numeric vector.

2. sortedDF <- mtcars[indexes,]

You use the indexes to reorder the rows of mtcars so that rows with lower mpg come first.

The result is a new data frame called sortedDF where the cars are sorted from lowest to highest mpg.

3. str(sortedDF)

This shows the structure of the sorted data frame.

It gives:

The number of observations (32) and variables (11)

Sorting a DataFrame the Tidyverse way:

```r
sortedDF <- mtcars %>%
               arrange(mpg)

glimpse(sortedDF)
```

```
## Rows: 32
## Columns: 11
## $ mpg  <dbl> 10.4, 10.4, 13.3, 14.3, 14.7, 15.0, 15.2, 15.2, 15.5, 15.8, 16.4,~
## $ cyl  <dbl> 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 6, 6, 8, 6, 8, 6, 6, 6, 6, 4,~
## $ disp <dbl> 472.0, 460.0, 350.0, 360.0, 440.0, 301.0, 275.8, 304.0, 318.0, 35~
## $ hp   <dbl> 205, 215, 245, 245, 230, 335, 180, 150, 150, 264, 180, 180, 123, ~
## $ drat <dbl> 2.93, 3.00, 3.73, 3.21, 3.23, 3.54, 3.07, 3.15, 2.76, 4.22, 3.07,~
## $ wt   <dbl> 5.250, 5.424, 3.840, 3.570, 5.345, 3.570, 3.780, 3.435, 3.520, 3.~
## $ qsec <dbl> 17.98, 17.82, 15.41, 15.84, 17.42, 14.60, 18.00, 17.30, 16.87, 14~
## $ vs   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1,~
## $ am   <dbl> 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1,~
## $ gear <dbl> 3, 3, 3, 3, 3, 5, 3, 3, 3, 5, 3, 3, 4, 3, 3, 4, 3, 5, 4, 4, 3, 4,~
```

```
## $ carb <dbl> 4, 4, 4, 4, 4, 8, 3, 2, 2, 4, 3, 3, 4, 1, 2, 4, 2, 6, 4, 4, 1, 2,~
```

Step-by-Step Explanation:

1. mtcars %>% arrange(mpg)

mtcars: the dataset

%>%: pipe operator — passes the data to the next function

arrange(mpg): sorts the rows in ascending order by the mpg column (miles per gallon)

So this line creates a new data frame (sortedDF) where:

Cars are ordered from lowest to highest mpg.

2. glimpse(sortedDF)

glimpse() is a tidyverse function from dplyr.

It shows a horizontal summary of the dataset:

The number of rows and columns

Each column name, its data type (like for numeric), and a sample of values.